

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет информатика и системы управления
Кафедра системы обработки информации и управления

Курс «Парадигмы и конструкции языков программирования»
Отчет по рубежному контролю №2
Вариант Б6

Выполнил:
студент группы ИУ5-32Б:
Кунев В.
Подпись и дата:

Проверил:
преподаватель каф. ИУ5
Гапанюк Ю.Е.
Подпись и дата:

Москва, 2025 г.

Постановка задачи

Рубежный контроль представляет собой разработку тестов на языке Python.

- 1) Проведите рефакторинг текста программы рубежного контроля №1 таким образом, чтобы он был пригоден для модульного тестирования.
- 2) Для текста программы рубежного контроля №1 создайте модульные тесты с применением TDD - фреймворка (3 теста).

Текст программы

``rk1_solution.py`:`

```
"""
```

```
Решение Рубежного контроля №1 по курсу ПИК ЯП (Отрефакторенное).
```

```
Вариант предметной области: 6 (Дом, Улица)
```

```
Вариант запросов: Б
```

```
Логика каждого запроса вынесена в отдельные функции
```

```
(solve_b1, solve_b2, solve_b3) для возможности модульного тестирования.
```

```
"""
```

```
from operator import itemgetter
```

```
class House:
```

```
    """Представление дома.
```

```
    Attributes:
```

```
        identifier (int): Уникальный идентификатор дома.
```

```
        number (str): Номер дома (например, '10a', '22').
```

```
        population (int): Количество жильцов в доме.
```

```
        street_id (int): Внешний ключ, ссылающийся на ID улицы (для связи 1:M).
```

```
    """
```

```
    def __init__(self, identifier: int, number: str, population: int, street_id: int):
```

```
        """Инициализирует экземпляр дома.
```

```
        Args:
```

```
            identifier (int): Уникальный идентификатор дома.
```

```
            number (str): Номер дома.
```

```
            population (int): Количество жильцов.
```

```
            street_id (int): ID связанной улицы.
```

```
        """
```

```
        self.identifier = identifier
```

```
        self.number = number
```

```
        self.population = population
```

```
        self.street_id = street_id
```

```
class Street:
```

```
    """Представление улицы.
```

```
    Attributes:
```

```
        identifier (int): Уникальный идентификатор улицы.
```

```
        name (str): Название улицы (например, 'ул. Ленина').
```

```
    """
```

```

def __init__(self, identifier: int, name: str):
    """Инициализирует экземпляр улицы.

    Args:
        identifier (int): Уникальный идентификатор улицы.
        name (str): Название улицы.
    """
    self.identifier = identifier
    self.name = name

class HouseStreet:
    """Ассоциативная сущность для связи "многие-ко-многим".

    Attributes:
        street_id (int): Внешний ключ, ссылающийся на ID улицы.
        house_id (int): Внешний ключ, ссылающийся на ID дома.
    """

    def __init__(self, street_id: int, house_id: int):
        """Инициализирует связь М:М.

        Args:
            street_id (int): ID связанной улицы.
            house_id (int): ID связанного дома.
        """
        self.street_id = street_id
        self.house_id = house_id

# --- Тестовые данные ---
streets: list[Street] = [
    Street(1, "ул. Ленина"),
    Street(2, "ул. Пушкина"),
    Street(3, "просп. Мира"),
]
houses: list[House] = [
    House(1, "10а", 50, 1),
    House(2, "12", 120, 1),
    House(3, "5а", 80, 2),
    House(4, "76", 200, 3),
    House(5, "22", 75, 2),
]
houses_streets: list[HouseStreet] = [
    HouseStreet(1, 1),
    HouseStreet(2, 1),
    HouseStreet(1, 2),
    HouseStreet(2, 3),
    HouseStreet(3, 4),
    HouseStreet(2, 5),
]

```

```
# --- Функции с логикой запросов ---
```

```
def solve_b1(
    one_to_many_data: list[tuple[str, int, str]]
) -> list[tuple[str, int, str]]:
    """
    Решение Задачи Б1:
    Вывести список всех связанных домов и улиц (1:M),
    отсортированный по домам (номеру дома).

    Args:
        one_to_many_data (list[tuple[str, int, str]]): Список кортежей
            (Номер дома, Кол-во жильцов, Название улицы).

    Returns:
        list[tuple[str, int, str]]: Тот же список, отсортированный
            по номеру дома (первый элемент кортежа).
    """
    return sorted(one_to_many_data, key=itemgetter(0))

def solve_b2(
    streets_data: list[Street], one_to_many_data: list[tuple[str, int, str]]
) -> list[tuple[str, int]]:
    """
    Решение Задачи Б2:
    Вывести список улиц с количеством домов в каждой (1:M),
    отсортированный по количеству домов.

    Args:
        streets_data (list[Street]): Список объектов Street (для итерации).
        one_to_many_data (list[tuple[str, int, str]]): Список кортежей
            (Номер дома, Кол-во жильцов, Название улицы) из соединения 1:M.

    Returns:
        list[tuple[str, int]]: Список кортежей (Название улицы, Кол-во домов),
            отсортированный по количеству домов (второй элемент кортежа).
    """
    res_unsorted: list[tuple[str, int]] = []
    for s in streets_data:
        s_houses: list[tuple[str, int, str]] = [
            item for item in one_to_many_data if item[2] == s.name
        ]

        if len(s_houses) > 0:
            s_count: int = len(s_houses)
            res_unsorted.append((s.name, s_count))

    return sorted(res_unsorted, key=itemgetter(1))
```

```

def solve_b3(
    many_to_many_data: list[tuple[str, int, str]]
) -> list[tuple[str, str]]:
    """
    Решение Задачи Б3:
    Вывести список всех домов, у которых номер заканчивается на «а» (М:М),
    и названия их улиц.

    Args:
        many_to_many_data (list[tuple[str, int, str]]): Список кортежей
            (Номер дома, Кол-во жильцов, Название улицы) из М:М соединения.

    Returns:
        list[tuple[str, str]]: Отфильтрованный список кортежей
            (Номер дома, Название улицы), где номер дома оканчивается на 'а'.
    """
    return [
        (h_num, s_name)
        for h_num, _, s_name in many_to_many_data
        if h_num.endswith("a")
    ]

def main():
    """
    Основная функция программы.

    Выполняет подготовку данных (join) и решает три задачи варианта 'Б',
    вызывая тестируемые функции.
    """

    # --- Подготовка данных (Join) ---

    one_to_many: list[tuple[str, int, str]] = [
        (h.number, h.population, s.name)
        for s in streets
        for h in houses
        if h.street_id == s.identifier
    ]
    many_to_many_temp: list[tuple[str, int, int]] = [
        (s.name, hs.street_id, hs.house_id)
        for s in streets
        for hs in houses_streets
        if s.identifier == hs.street_id
    ]
    many_to_many: list[tuple[str, int, str]] = [
        (h.number, h.population, s_name)
        for s_name, s_id, h_id in many_to_many_temp
        for h in houses
    ]

```

```

        if h.identifier == h_id
    ]

# --- Выполнение запросов ---

print("Задание Б1")
res_1 = solve_b1(one_to_many)
print(res_1)

print()
print("Задание Б2")
res_2 = solve_b2(streets, one_to_many)
print(res_2)

print()
print("Задание Б3")
res_3 = solve_b3(many_to_many)
print(res_3)

if __name__ == "__main__":
    main()

```

`test\_rk1\_solution.py`:

```

"""
Модульные тесты для Рубежного контроля №1.
"""

import unittest
from rk1_solution import (
    House,
    Street,
    HouseStreet,
    solve_b1,
    solve_b2,
    solve_b3,
)

class TestRK1Queries(unittest.TestCase):
    """Тест-кейс для 3-х запросов варианта Б."""

    def setUp(self):
        """
        Настройка тестового окружения (Fixtures).

```

Создает наборы тестовых данных (улицы, дома, связи) и подготавливает входные данные ('one_to_many', 'many_to_many') для каждого тестового метода.

```
"""
self.streets: list[Street] = [
    Street(1, "ул. Ленина"),
    Street(2, "ул. Пушкина"),
    Street(3, "просп. Мира"),
]
self.houses: list[House] = [
    House(1, "10а", 50, 1),
    House(2, "12", 120, 1),
    House(3, "5а", 80, 2),
    House(4, "76", 200, 3),
    House(5, "22", 75, 2),
]
self.houses_streets: list[HouseStreet] = [
    HouseStreet(1, 1),
    HouseStreet(2, 1),
    HouseStreet(1, 2),
    HouseStreet(2, 3),
    HouseStreet(3, 4),
    HouseStreet(2, 5),
]

self.one_to_many = [
    (h.number, h.population, s.name)
    for s in self.streets
    for h in self.houses
    if h.street_id == s.identifier
]

many_to_many_temp = [
    (s.name, hs.street_id, hs.house_id)
    for s in self.streets
    for hs in self.houses_streets
    if s.identifier == hs.street_id
]
self.many_to_many = [
    (h.number, h.population, s_name)
    for s_name, s_id, h_id in many_to_many_temp
    for h in self.houses
    if h.identifier == h_id
]
```

```
def test_solve_b1(self):
```

```
    """Тестирование Задачи Б1 (сортировка домов по номеру).
```

Проверяет, что функция solve_b1 корректно сортирует список 1:М по номеру дома.

```
    """
```



```

expected = [
    ("10a", 50, "ул. Ленина"),
    ("12", 120, "ул. Ленина"),
    ("22", 75, "ул. Пушкина"),
    ("5a", 80, "ул. Пушкина"),
    ("76", 200, "просп. Мира"),
]

result = solve_b1(self.one_to_many)

self.assertEqual(result, expected)

def test_solve_b2(self):
    """Тестирование Задачи Б2 (количество домов по улицам).

    Проверяет, что функция solve_b2 корректно группирует дома по улицам,
    подсчитывает их количество и сортирует результат по количеству домов.
    """
    expected = [
        ("просп. Мира", 1),
        ("ул. Ленина", 2),
        ("ул. Пушкина", 2),
    ]

    result = solve_b2(self.streets, self.one_to_many)

    self.assertEqual(result, expected)

def test_solve_b3(self):
    """Тестирование Задачи Б3 (дома на 'а' в М:М).

    Проверяет, что функция solve_b3 корректно фильтрует список М:М,
    оставляя только дома с номером, оканчивающимся на 'а'.
    """
    expected = [
        ("10a", "ул. Ленина"),
        ("10a", "ул. Пушкина"),
        ("5a", "ул. Пушкина"),
    ]

    result = solve_b3(self.many_to_many)

    self.assertEqual(result, expected)

if __name__ == "__main__":
    unittest.main()

```

Скриншот работы приложения

```
(.venv) cunev@ROGStrixG814JIR:~/VSCode/ParadigmsAndConstructsOfProgrammingLanguages/rk2$ pytest
===== test session starts =====
platform linux -- Python 3.12.3, pytest-8.4.1, pluggy-1.6.0
rootdir: /home/cunev/VSCode/ParadigmsAndConstructsOfProgrammingLanguages/rk2
collected 3 items

test_rk1_solution.py ... [100%]

===== 3 passed in 0.01s =====
(.venv) cunev@ROGStrixG814JIR:~/VSCode/ParadigmsAndConstructsOfProgrammingLanguages/rk2$ python test_rk1_solution.py
...
-----
Ran 3 tests in 0.000s

OK
```

Рисунок 1. Вывод результатов программы